TP-Colle 20 (Version Mines)

Génération de textes à partir de N-grammes

Le code source pour ce TP-Colle est à récupérer sur la page Cahier-de-Prépa du cours, ou sur le réseau du lycée.

Récupération du code source

Fichiers fournis : On fournira pour ce TP-Colle :

- un fichier `cid.txt` utilisé pour les tests dans la section II.
- un fichier `mots.ml` dans lequel on écrira le code demandé dans la section IV.
- une archive `renaud.zip`, contenant des fichiers `.txt` qui seront utilisés pour les tests dans la section IV
- un fichier `parser.ml`, contenant du code permettant de lire les fichiers se trouvant dans l'archive précédente.

Compilation et exécution :

- Pour compiler un projet en OCaml dans lequel on souhaite exécuter le code d'un fichier `main.ml`, utilisant du code présent dans d'autres fichiers `dep1.ml` et `dep2.ml`, on peut :
  - compiler en une fois, à l'aide d'`ocamlc` ou `ocamlopt` :

```
ocamlc -o nom_executable dep1.ml dep2.ml main.ml
ocamlopt -o nom_executable dep1.ml dep2.ml main.ml
```

  - compiler d'abord les fichiers de dépendance comme des modules, qui seront utilisables par le fichier principal, à l'aide d'`ocamlc` ou `ocamlopt` :

```
ocamlc -g dep1.ml                          ocamlopt -g dep1.ml
ocamlc -g dep2.ml                          ocamlopt -g dep2.ml
ocamlc -o nom_executable dep1.cmo dep2.cmo ocamlopt -o nom_executable dep1.cmx
main.ml                                    dep2.cmx main.ml
```

- Pour compiler un projet en C dans lequel on souhaite exécuter le code d'un fichier `main.c`, utilisant du code présent dans d'autres fichiers `dep1.c` et `dep2.c`, on peut :
  - compiler en une fois, à l'aide de`gcc` :

```
gcc -o nom_executable dep1.c dep2.c main.c
```

  - compiler d'abord les fichiers de dépendance en fichiers `.o`, qui seront utilisables par le fichier principal, à l'aide d'`gcc` ou `ocamlopt` :

```
gcc -c dep1.c
gcc -c dep2.c
gcc -o nom_executable dep1.o dep2.o main.c
```

  - on peut également ajouter un certain nombre d'options de compilation : `-Wall` et `-Wextra` pour les warnings, ainsi que `-fsanitize=address,undefined` à la fin de la ligne de compilation si on souhaite détecter des fuites mémoire.

- si un `Makefile` est fourni, il suffit de taper `make` (éventuellement suivi d'un nom de règle) pour compiler le projet.
- Pour exécuter, il suffit de taper `./nom_executable`.

Modalités d'évaluation

On se basera sur ce qui suit pour l'évaluation du TP-Colle :

- **Compte-rendu du TP** : pendant l'avancement du TP, rédigez un compte-rendu qui consignera le traitement de **toutes** les questions de l'épreuve, écrites sur le sujet et posées par l'examinateur.
- **Echange oraux** : l'examinateur circulera entre les étudiants évalués, pour vérifier la progression, et procéder à un échange autour du sujet. Cet échange peut-être mené de manière assez libre par l'examinateur, portant aussi bien sur le code que sur d'autres aspects du sujet.
- **Code** : le code produit sera ramassé (envoyez-le à `joshua.peignier@ac-strasbourg.fr`), mais ne sera examiné après l'oral qu'en cas de nécessité.

Introduction

On souhaite développer un système permettant de prédire la suite d'une séquence de caractères. Ce genre de système est utilisé pour compléter les messages mails ou SMS par exemple. L'utilisateur commence à taper son message (une séquence de caractères) et le système propose de le compléter.

L'approche considérée est basée sur les N-grammes par apprentissage sur un texte préexistant. Un N-gramme est une séquence de N caractères consécutifs apparaissant dans le texte. Cette approche est expliquée sur le texte suivant (un réel apprentissage nécessite un texte bien plus grand):

`Bonjour,␣comment␣allez-vous␣?␣Ca␣va,␣ca␣va␣aller␣bien␣mieux.`

L'objectif de l'approche est:

1. de construire les N-grammes contenus dans le texte;
2. d'identifier les caractères qui leur succèdent en calculant les probabilités associées.

La première étape nécessite de choisir la valeur de N, la taille des N-grammes. Dans l'exemple précédent, le texte contient, par ordre d'apparition:

- les 1-grammes: `"B"`, `"o"`, `"n"`, ...,`"r"`, `","`, `"␣"`, `"c"`, ...
- les 2-grammes: `"Bo"`, `"on"`, `"nj"`, ...
- les 3-grammes: `"Bon"`, `"onj"`, `"njo"`, ...
- etc

La deuxième étape vise à identifier les caractères qui suivent chaque N-gramme, qu'on appellera les *successeurs*, et compter leur occurrences pour connaître leur probabilité d'apparition. Par exemple:

- le 1-gramme `"a"` peut être suivi par les caractères `'l'`, `'␣'` et `','` avec respectivement 2, 3 et 1 occurrences, donc avec les probabilités $\frac{2}{6}$, $\frac{3}{6}$ et $\frac{1}{6}$;
- le 1-gramme `"u"` peut être suivi par les caractères `'r'` et `'s'` avec respectivement 1 et 1 occurrences, donc avec les probabilités $\frac{1}{2}$ et $\frac{1}{2}$;
- le 2-gramme `"le"` peut être suivi par les caractères `'z'` et `'r'` avec respectivement 1 et 1 occurrences, donc avec les probabilités $\frac{1}{2}$ et $\frac{1}{2}$;

En choisissant une taille de N-grammes, c'est-à-dire en fixant N, on dispose alors de probabilités conditionnelles qui permettent de prédire le caractère qui fait suite à une séquence de caractères donnée.

I Modèle de 1-grammes

Cette partie doit être réalisée en C. Aucun code source n'est fourni, on travaillera dans un fichier vierge.

Dans une version simplifiée de modèle de 1-grammes, le caractère à prédire après un autre sera toujours le même, à savoir le plus fréquent parmi les successeurs. En cas d'égalité de nombres d'occurrences entre deux successeurs, on choisira arbitrairement le premier dans l'ordre alphabétique, à savoir celui qui a le plus petit code associé dans l'alphabet ASCII (contenant 128 caractères). Ainsi, le modèle n'a pas à garder en mémoire l'ensemble des successeurs pour chaque caractère, mais seulement celui qui sera prédit.

En reprenant les exemples précédents, le modèle prédira que:

- le 1-gramme `"a"` sera suivi par le caractère `'␣'` qui est le plus fréquent parmi les successeurs;
- le 1-gramme `"u"` sera suivi par le caractère `'r'`, qui apparaît aussi fréquemment que `'s'` mais se trouve avant `'s'` dans l'ordre alphabétique.

On implémente un modèle de 1-grammes par un tableau d'entiers `M`, de taille 128, tel que si `c` est un caractère de code `k`, alors `M[k]` vaut:

- le code du caractère à prédire si `c` possède des successeurs;
- le code 0 du caractère nul `'\0'` sinon.

Dans la suite, on posera $K = 128$ la taille de l'alphabet ASCII. Même si on travaillera de manière fixe avec l'alphabet ASCII, on exprimera également les complexités **en fonction de** K, pour étendre le raisonnement sur des alphabets plus grands.

Question 1. Écrire une fonction `int plus_frequent_successeur(char c, char* chaine)` qui prend en argument un caractère `c` et une chaîne de caractère et renvoie le code du caractère qui apparaît le plus fréquemment dans la chaîne comme successeur de `c`, ou le plus petit code en cas d'égalité. La fonction renverra le code du caractère nul si `c` n'apparaît pas dans la chaîne.

Question 2. En déduire une fonction `int *init_modele(char* chaine)` qui construit et renvoie un modèle de 1-gramme à partir d'un texte d'apprentissage donné en argument.

Question 3. Quelle est la complexité temporelle de la fonction précédente ? Peut-on faire mieux ? On ne demande pas d'implémenter une éventuelle solution mais de la décrire succinctement à l'examinateur.

Question 4. Écrire une fonction `int** matrice_confusion(int* M, char* test)` qui prend en argument un modèle de 1-gramme et une chaîne de caractères `test` et renvoie une matrice de confusion de taille $K \times K$ comparant les prédictions données par le modèle et le successeurs de chaque caractère dans la chaîne de test.

Question 5. Évaluer le modèle de prédiction entraîné sur la phrase donnée en exemple à la page précédente, en calculant le pourcentage d'erreurs à partir de la matrice de confusion obtenue pour la phrase de test suivante :

`Bonjour,␣ca␣va␣bien␣?␣Oui␣!␣Bien␣mieux,␣et␣vous,␣ca␣va␣?`

II Modèle de N-grammes

Cette partie doit être réalisée en OCaml. Aucun code source n'est fourni, on travaillera dans un fichier vierge.

Par rapport au modèle précédent, on propose deux changements:

- la prédiction se fait non plus à partir du dernier 1-gramme lu, mais du dernier N-gramme, ou le dernier $(N-1)$-gramme si le N-gramme n'a pas de successeur, ou du dernier $(N-2)$-gramme si le $(N-1)$-gramme n'a pas de successeur, ... en allant si besoin jusqu'au 0-gramme (la chaîne vide), dont les successeurs sont l'ensemble des lettres du texte d'apprentissage;
- la prédiction n'est plus déterministe, mais choisit aléatoirement un successeur pour un N-gramme en fonction de la probabilité associée.

Par exemple:

- le modèle prédira que le 1-gramme `"a"` sera suivi par le caractère `'l'` avec la probabilité $\frac{2}{6}$, `'␣'` avec probabilité $\frac{3}{6}$ et `','` avec probabilité $\frac{1}{6}$;
- le 3-gramme `"ple"` n'a pas de successeur, donc le modèle utilisera le 2-gramme `"le"` pour faire la prédiction, qui sera le caractère `'r'` avec probabilité $\frac{1}{2}$ et `'z'` avec probabilité $\frac{1}{2}$;
- le 2-gramme `"ay"` n'a pas de successeur, et le 1-gramme `"y"` non plus, donc le modèle utilisera le 0-gramme pour faire la prédiction, qui sera le caractère `'B'` avec probabilité $\frac{1}{60}$, le caractère `'o'` avec probabilité $\frac{4}{60}$, le caractère `'n'` avec probabilité $\frac{3}{60}$, ...

Dans cette partie, un modèle de N-grammes doit contenir, pour chaque k-gramme du texte d'apprentissage, avec $k \leq N$, l'ensemble de ses successeurs et du nombre d'occurrences correspondant, ou des probabilités associées.

Question 6. Décrire une structure de données pour encoder un modèle de N-grammes avec un N donné. Cette structure doit permettre de retrouver les modèles dont la taille est inférieure à N. Par exemple, un modèle de 3-grammes devra contenir les successeurs des 3-grammes, mais également des 2-grammes, des 1-grammes et du 0-gramme.
Puis implémenter la dite structure, **après l'avoir faite valider par l'examinateur**.

Question 7. Écrire une fonction qui construit et renvoie un modèle de N-grammes à partir d'un texte d'apprentissage et d'un entier N donnés en argument.

Question 8. Écrire une fonction qui prend en argument un modèle et un N-gramme et renvoie un caractère prédit par le modèle selon le principe décrit précédemment.

Pour tester le modèle, on souhaite faire de la génération automatique de texte en *pseudo-français*, c'est-à-dire un texte dont les mots ressemblent à du français mais n'en sont pas forcément. L'idée est de partir d'une chaîne de caractères (vide ou non) et de prédire les caractères suivants, à chaque fois en considérant le dernier N-gramme, qui contient éventuellement les lettres déjà prédites. Voici par exemple un texte généré automatiquement à partir de la chaîne `"Bonjour"` en utilisant le modèle de la phrase donnée initialement en exemple, et pour $N = 2$:

```
Bonjour, ca allez-vour, ca va, ca aller bien mien mieux.-vous ? Ca va, commen mieux. va va aller
bieux.va va, commen mient allez-vous ? Ca va va, ca va va va, ca va va va, comment aller bieux
```

Question 9. Écrire une fonction qui prend en argument un modèle de N-grammes, une chaîne de caractères et une taille cible et génère un texte aléatoire de taille cible à partir de la chaîne, en utilisant le modèle de prédiction, selon le principe décrit précédemment.

Question 10. Le fichier `cid.txt` contient un extrait du *Cid* de Corneille. Utiliser ce texte pour faire de la génération automatique pour différentes valeurs de N.

III Programmation concurrente

Cette partie doit être réalisée en OCaml dans le fichier précédent.

La construction d'un modèle de N-grammes peut prendre du temps surtout si le texte d'entrée est très long. Pour optimiser la construction, on souhaite mettre en place une approche exploitant la concurrence, avec plusieurs fils d'exécution.

Question 11. Proposer une approche pour répartir la construction du modèle sur plusieurs fils.

Question 12. Implémenter cette approche.

IV N-grammes de mots

Cette partie doit être réalisée en Ocaml. On travaillera dans le fichier `mots.ml` fourni pour écrire le code demandé.

On souhaite étendre le modèle vu dans la section II, mais cette fois-ci, en ne considérant plus comme unité lexicale des caractères (de type `char`), mais des chaînes (de type `string`) pouvant représenter des mots de la langue française, des nombres, des signes de ponctuation, etc... Ainsi, dans la suite :

- On travaillera désormais sur des objets d'un nouveau type `texte = string array`. Pour reprendre l'exemple du début, on le traduirait en OCaml en le `texte` suivant :

```
[|"Bonjour" ; "," ; "␣" ; "comment" ; "␣" ; "allez" ; "-" ; "vous" ; "␣" ; "?" ; "␣" ;
"Ca" ; "␣" ; "va" ; "," ; "␣" ; "ca" ; "␣" ; "va" ; "␣" ; "aller" ; "␣" ; "bien" ; "␣" ;
"mieux" ; "." |]
```

  Dans la suite, nos textes d'entraînement ne seront donc plus des `string`, mais des `texte`.

- on peut maintenant redéfinir un N-gramme comme étant un `texte` de longueur N
- on souhaite faire le même travail que dans la section II à partir de cette nouvelle définition. Par exemple, étudier le texte précédent et établir un modèle de prédiction de sorte :
  - le modèle prédira que le 2-gramme `[|"␣" ; "va"|]` sera suivi par la chaîne `","` avec la probabilité $\frac{1}{2}$, ou par la chaîne `"␣"` avec probabilité $\frac{1}{2}$.
  - le 3-gramme `[|"mieux" ; "," ; "␣"|]` n'a pas de successeur, donc le modèle utilisera le 2-gramme `[|"," ; "␣"|]` pour faire la prédiction, qui sera la chaîne `"comment"` avec la probabilité $\frac{1}{2}$, ou par la chaîne `"ca"` avec probabilité $\frac{1}{2}$.
  - le 2-gramme `[|"Salut" ; "!"|]` n'a pas de successeur, et le 1-gramme `[|"!"|]` non plus, donc le modèle utilisera le 0-gramme pour faire la prédiction, qui sera la chaîne `[|"Bonjour"|]` avec probabilité $\frac{1}{26}$, ou la chaîne `[|"va"|]` avec probabilité $\frac{1}{13}$, ou la chaîne `[|"␣"|]` avec probabilité $\frac{5}{13}$, ...

Question 13. Adapter la structure de données de la section précédente pour cette nouvelle définition de N-grammes, puis proposer un type pour implémenter la dite structure. **On le fera valider par l'examinateur avant de passer à la suite**. Créer également une fonction `create_modele()` qui renvoie un modèle vide, n'ayant encore subi aucun entraînement.

Question 14. Pour s'aider à faire des tests sur ce modèle significativement plus complexe, on fournit un certain nombre de fonctions d'affichage, prenant en entrée un flux sortant de type `out_channel` (ce peut être le flux obtenu en ouvrant en fichier en écriture avec `open_out`, ou ce peut être la constante `stdout` pour afficher dans le terminal). Compléter leur code.

Question 15. Écrire une fonction qui prend en entrée un modèle de N-grammes tel que défini précédement, l'enrichit à partir d'un texte d'apprentissage et d'un entier N donnés en argument.

Question 16. Écrire une fonction qui prend en argument un modèle et un N-gramme et renvoie une chaîne prédite comme successeur par le modèle selon le principe décrit précédemment.

Pour effectuer les tests, on peut maintenant :

- décompresser le fichier `renaud.zip` ; il contient plusieurs fichiers `.txt`, chacune contenant les paroles d'une chanson de Renaud[1]. Ces fichiers doivent figurer dans le même dossier que votre code.
- compiler le fichier `parser.ml` ; celui donne accès à une fonction `parse : string -> bool -> texte`, tel que l'appel `parse "fichier.txt" false`[2] renvoie un `string array` correspondant au texte contenu dans le fichier `fichier.txt`
- utiliser la constante `renaud` fournit en variable globale, qui est un tableau contenant les noms des fichiers présents dans `renaud.zip`

Question 17. Ecrivez le code nécessaire pour créer un modèle vide, l'enrichir à partir d'une ou plusieurs chansons de Renaud, et faire de la génération de tests avec différentes valeurs de N.

Annexes OCaml

Aléatoire

On peut tirer un entier pseudo-aléatoire entre 0 inclus et n exclu par la commande `Random.int n`. On doit alors exécuter `Random.self_init ()` pour utiliser une graine aléatoire.

[1] Artiste choisi totalement au hasard et sans aucun lien avec mes propres goûts musicaux.

[2] Si la lecture d'un fichier semble échouer, on pourra utiliser `parse "fichier.txt" true` pour activer le mode debug du parser : celui-ci écrira dans un fichier `debug` les opérations qu'il fait, et permettra de cerner si l'un des fichiers .txt fournis a un problème. Si cela doit arriver, ce sera généralement à cause de caractères accentués, qui ne sont pas représentés dans le type `char` en OCaml.

Caractères

`Char.code : char -> int` et `Char.chr : int -> char` permettent de faire la conversion entre caractère et code associé. La fonction `String.sub : string -> int -> int` est telle que l'appel `String.sub s deb taille` renvoie la sous-chaîne de `s` commençant à l'indice `deb` et de taille `taille`.

Dictionnaire

L'utilisation de dictionnaires peut se faire à l'aide du module `Hashtbl`. On dispose notamment des commandes :

- `Hashtbl.create : int -> ('a, 'b) Hashtbl.t` qui prend en argument un entier correspondant à une estimation du nombre de clés qui vont être associées dans la table (en pratique on peut simplement utiliser `1` car les tables sont redimensionnables dynamiquement) et crée une table vide ;
- `Hashtbl.add : ('a, 'b) Hashtbl.t -> 'a -> 'b -> unit` qui prend en argument une table, une clé et une valeur et rajoute une association ;
- `Hashtbl.mem : ('a, 'b) Hashtbl.t -> 'a -> bool` qui teste si une table contient une association pour une clé donnée ;
- `Hashtbl.find : ('a, 'b) Hashtbl.t -> 'a -> 'b` qui prend en argument une table et une clé et renvoie la valeur associée, ou lève l'exception `Not_found` si la clé n'a pas d'association ;
- `Hashtbl.iter : ('a -> 'b -> unit) -> ('a, 'b) t -> unit` qui prend en argument une fonction f et une table et applique la fonction f à chaque couple (clé, valeur) de la table.

Fichiers

La lecture dans un fichier peut se faire à l'aide des fonctions suivantes :

- `open_in : string -> in_channel` qui prend en argument une chaîne de caractère représentant le nom d'un fichier et renvoie un flux depuis lequel on peut recevoir des caractères.
- `input_line : in_channel -> string` qui prend en argument un flux, lit une ligne (terminant donc par un caractère `\n` et renvoie la chaîne de caractère correspondante.
- `close_in : in_channel -> unit` ferme un flux.

L'écriture dans un fichier peut se faire à l'aide des fonctions suivantes (on rappel que le la constante `stdout` de type `out_channel` correspond à la sortie sur le terminal) :

- `open_out : string -> out_channel` qui prend en argument une chaîne de caractère représentant le nom d'un fichier et renvoie un flux vers lequel on peut envoyer des caractères.
- `Printf.fprintf : out_channel -> string -> ... -> unit` qui prend en argument un flux, une chaîne de caractères formatée pour `printf` avec éventuellement ses arguments, et l'envoie dans le flux.
- `close_out : out_channel -> unit` ferme un flux.

Concurrence

L'utilisation des outils de synchronisation peut se faire à l'aide des commandes suivantes. Pour simplifier l'utilisation des sémaphores, on peut renommer le module avec : `module Sem = Semaphore.Counting`

- `Thread.create : ('a -> 'b) -> 'a -> Thread.t` prend en argument une fonction `f` et un argument `x` et renvoie un fil d'exécution qui exécute `f x`. **Attention** : la fonction `f` ne doit avoir qu'un seul argument. Si on souhaite la faire travailler avec plusieurs arguments, on peut utiliser un tuple ;
- `Thread.join : Thread.t -> unit` prend en argument un fil d'exécution et attend qu'il ait terminé son exécution ;

- `Mutex.create : unit -> Mutex.t` renvoie un nouveau mutex ;
- `Mutex.lock : Mutex.t -> unit` verrouille un mutex ;
- `Mutex.unlock : Mutex.t -> unit` déverrouille un mutex ;
- `Sem.make : int -> Sem.t` prend en argument un entier k et renvoie un sémaphore avec un compteur initialisé à k ;
- `Sem.acquire : Sem.t -> unit` acquière un sémaphore (c'est-à-dire décrémente le compteur) ;
- `Sem.release : Sem.t -> unit` libère un sémaphore (c'est-à-dire incrémente le compteur).

Attention, il est nécessaire d'utiliser l'option de compilation suivante: `-I +threads unix.cma threads.cma`.

On peut également utiliser un interpréteur OCaml au lieu de compiler, auquel cas il sera nécessaire d'exécuter les commandes suivantes :

```
#directory "+threads";;
#load "unix.cma";;
#load "threads.cma";;
```